

Q-1 Find the order of growth for solution using recursive tree & verify using Master's theorem

Recor

$$i) T(n) = \begin{cases} 1 & n=1 \\ 4T(\frac{n}{4}) + n & n>1 \end{cases}$$

Total cost
 n

level no. of nodes

0

1

n

$T(n)$

1

4

$T(\frac{n}{4})$

$T(\frac{n}{4})$

$T(\frac{n}{4})$

$T(\frac{n}{4})$

$4 \times \frac{n}{4} = n$

2

4^2

$T(\frac{n}{4^2})$

$T(\frac{n}{4^2})$

$T(\frac{n}{4^2})$

$T(\frac{n}{4^2})$

$T(\frac{n}{4^2})$

$4^2 \times \frac{n}{4^2} = n$

i

4^i

$T(\frac{n}{4^i})$

$T(\frac{n}{4^i})$

$T(\frac{n}{4^i})$

$T(\frac{n}{4^i})$

$T(\frac{n}{4^i})$

$T(\frac{n}{4^i})$

$T(\frac{n}{4^i})$

h

4^h

$T(\frac{n}{4^h})$

$T(\frac{n}{4^h})$

$T(\frac{n}{4^h})$

$T(\frac{n}{4^h})$

$T(\frac{n}{4^h})$

$T(\frac{n}{4^h})$

$4^h \times \frac{n}{4^h} = n$

Find the value of h

$$\frac{n}{4^h} = 1$$

$$h = \log_4 n$$

$$T(n) = \sum_{i=1}^h n = \sum_{i=1}^{\log_4 n} n$$

$$= n + n + \dots \quad h \text{ times}$$

$$= n \log_4 n$$

$$= O(n) = n \log n$$

Solving by Masters Theorem

As we know, the recursive eqⁿ should be in the form,

$$T(n) = a T\left(\frac{n}{b}\right) + O(n^k \log^p n)$$

for the given problem

$$T(n) = 4T\left(\frac{n}{4}\right) + n$$

$$a=4 \quad b=4 \quad k=1 \quad p=0$$

$$\log_a b = \log_4 4 = 1 \neq$$

hence $\log_a b = 1 = k$

and since $p > -1$

$$T(n) = O(n^{\log_a b} \log^{p+1} n)$$

$$= O(n^1 \log^1 n)$$

$$= O(n \log n)$$

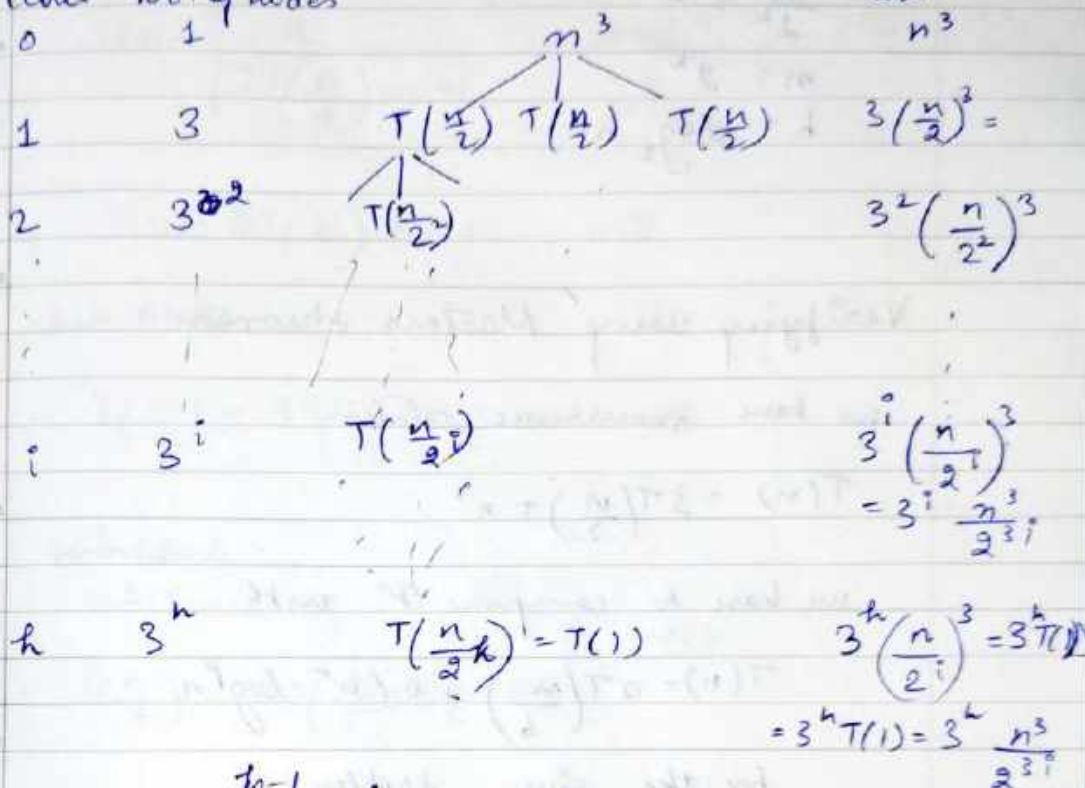
Hence Verified

The order of growth is $O(n \log n)$

ii) $T(n) = 3T\left(\frac{n}{2}\right) + n^3$

level No. of nodes

Cost
 n^3



$$T(n) = n^3 \sum_{i=0}^{h-1} \frac{3^i}{(2^3)^i} + 3^h (T(1))$$

$$\frac{n}{2^h} = 1$$

$$h = \log_2 n$$

$$= n^3 \sum_{i=0}^{h-1} \left(\frac{3}{8}\right)^i + 3^h T(1)$$

$$= n^3 \left[1 + \frac{3}{8} + \left(\frac{3}{8}\right)^2 + \dots \right] + 3^{\log_2 n}$$

$$= n^3 \left(\frac{1}{1 - \frac{3}{8}} \right) + n^{\log_2 3}$$

$$T(n) = n^3 \left(\frac{8}{5} \right) + n^{\log_2 3}$$

$$T(n) = O(n^3)$$

Order of growth = $O(n^3)$

To calculate h

$$\frac{n}{2^h} = 1$$

$$n = 2^h$$

$$h = \log_2 n$$

Verifying using Masters theorem.

we have recurrence relation

$$T(n) = 3T\left(\frac{n}{2}\right) + n^3$$

we have to compare it with.

$$T(n) = aT\left(\frac{n}{b}\right) + O(n^k \log^p n)$$

for the given problem.

$$a=3 \quad b=2 \quad k=3 \quad p=0$$

$$\text{Now } \log_b a = \log_2 3$$

$$\log_2 3 < 3 \quad (\log_b a < k)$$

$$\text{and } p \geq 0 \quad p=0$$

$$T(n) = O(n^k \log^p n)$$

$$= O(n^3 \log^0 n)$$

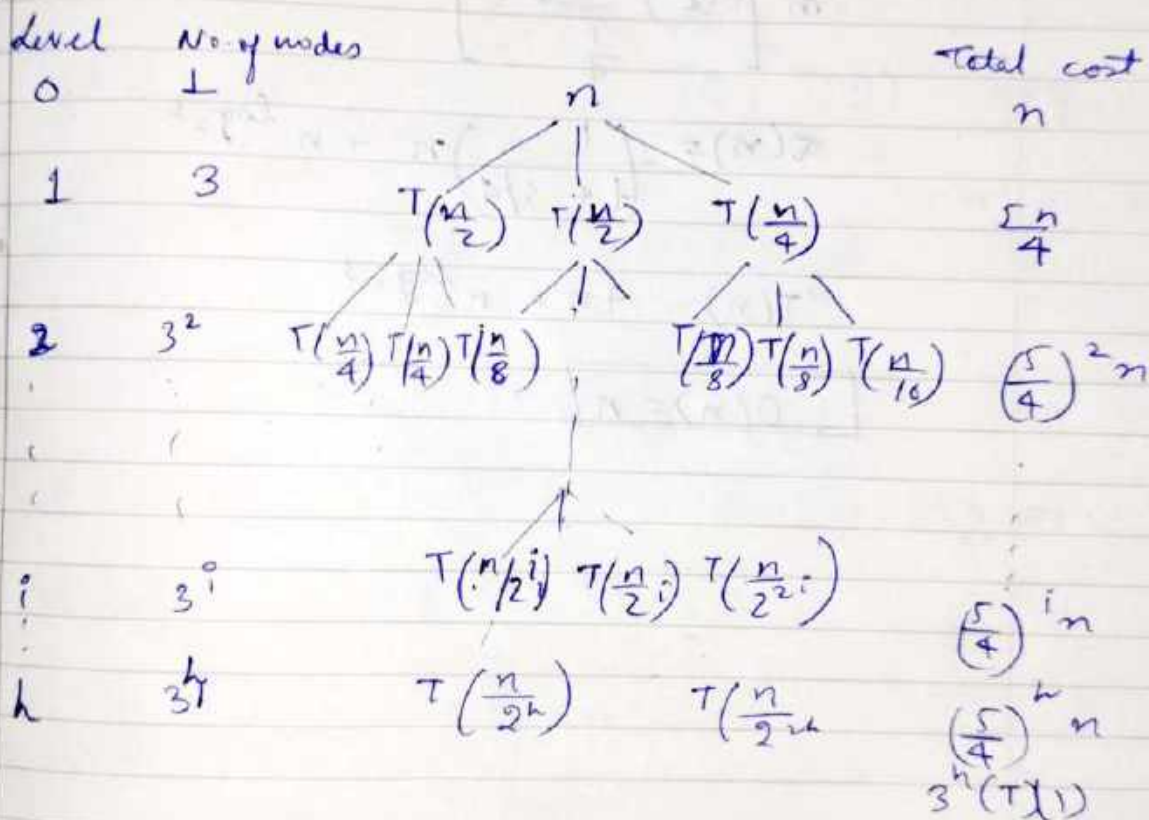
$$O(n^3)$$

Hence Verified.

3. Find the order of growth for solutions of the following recurrences using recursive tree method.

$$T(n) = \begin{cases} 1 & n=1 \\ 2T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + n & \text{when } n > 1 \end{cases}$$

$$T(n) = \begin{cases} 1 & n=1 \\ 2T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + n \end{cases}$$



Finding the value of h

We consider $\frac{n}{2^h}$ to be the base case where n since it has greater value.

$$\text{So, } T(1) = T\left(\frac{n}{2^h}\right) \Rightarrow \frac{n}{2^h} = 1 \quad 2^h = n$$

$$h = \log_2 n$$

$$T(n) = 1 \sum_{i=1}^{h-1} \left(\frac{5}{4}\right)^i n + 3^h T(1)$$

$$= n \left[1 + \frac{5}{4} + \dots + \left(\frac{5}{4}\right)^{h-1} \right] + 3^{\log_2 n}$$

$$= n \left[\frac{\left(\frac{5}{4}\right)^{h-1+1} - 1}{\frac{5}{4} - 1} \right] + n^{\log_2 3}$$

$$n \left[\frac{\left(\frac{5}{4}\right)^h - 1}{\frac{1}{4}} \right]$$

$$T(n) = \left(\frac{1}{1 + 5/4} \right) n + n^{\log_2 3}$$

$$T(n) = 4n + n^{\log_2 3}$$

$$\boxed{O(n) = n}$$

The order of growth is $O(n) = n$

4. Find the Huffman codeword of characters given below.

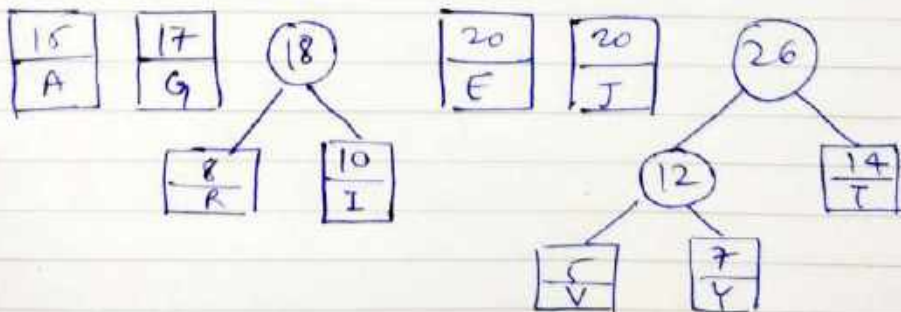
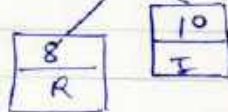
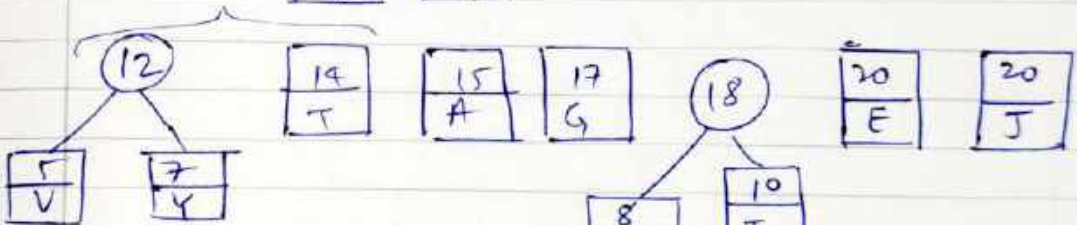
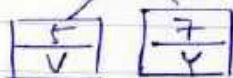
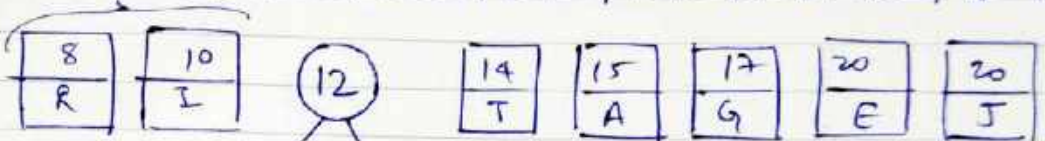
Char A I Y V R E G J T
freq 15 10 7 5 8 20 17 20 14

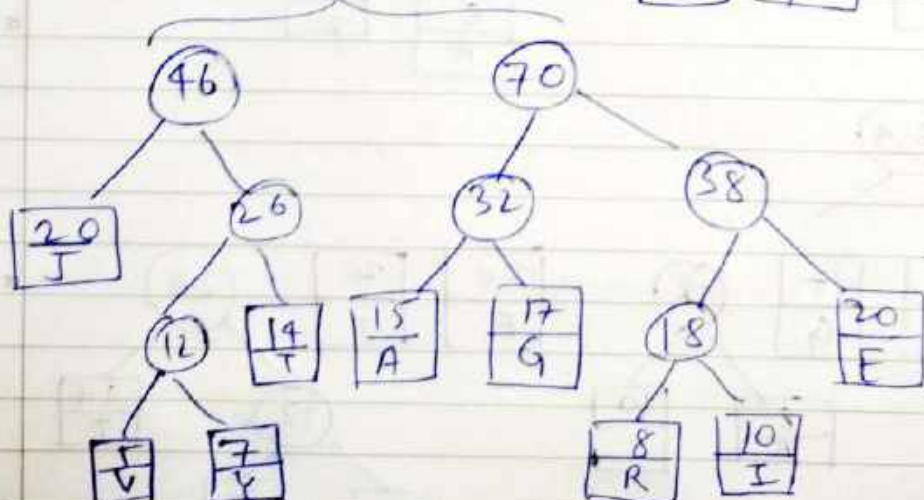
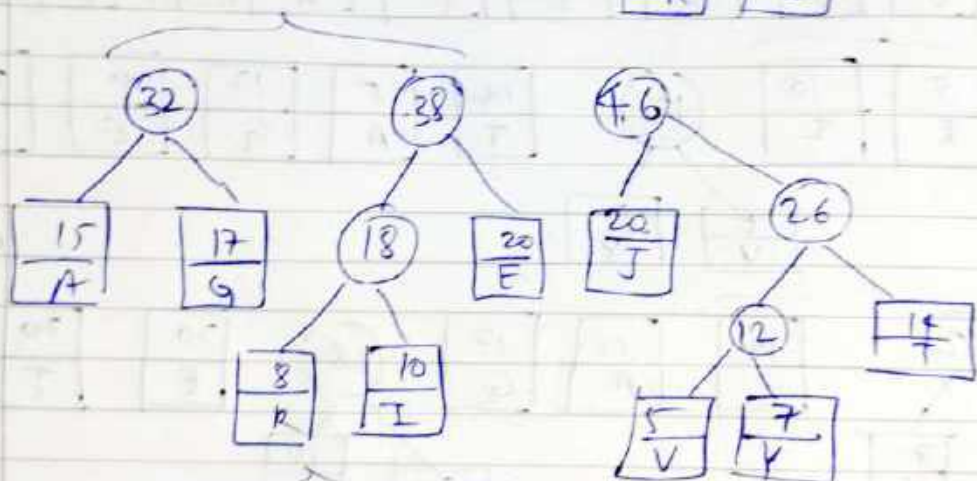
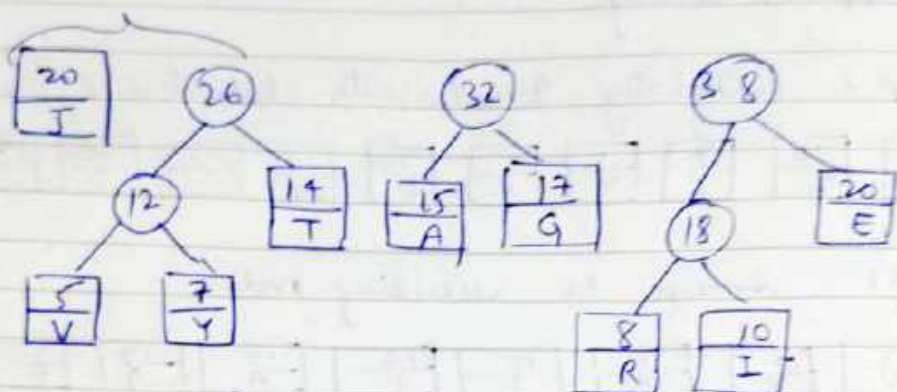
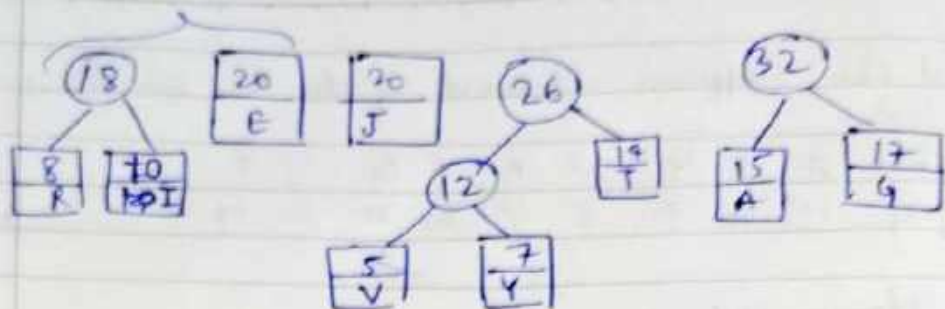
Huffman Coding

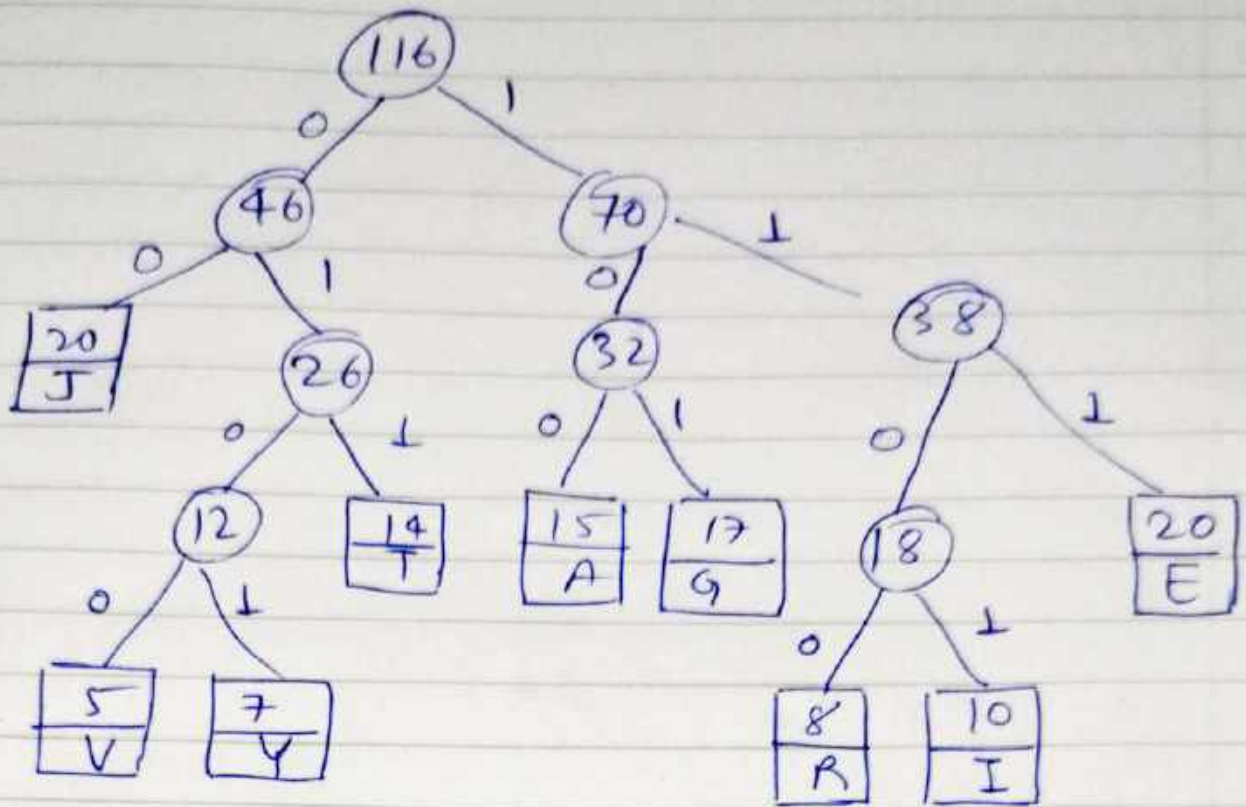
Step 1 Initializing 9 nodes with characters & their freq.



Step 2. Arrange in ascending order.







Characters

Huffman Codes.

A

100

I

1101

Y

0101

V

0100

R

1100

E

111

G

101

J

00

T

011

Algorithm of Huffman-coding

Huffman(c)

$n = |c|$

$Q = c$

for $i = 1$ to $n - 1$

 allocate a new node z

$z \cdot \text{left} = x = \text{Extract-min}(Q)$

$z \cdot \text{right} = y = \text{Extract-min}(Q)$

$z \cdot \text{freq} = x \cdot \text{freq} + y \cdot \text{freq}$

 Insert(Q, z)

return $\text{Extract-min}(Q)$

// return root of the tree.

- 5.) Solve the following
 a) The weights and values of divisible items are given in the table below. The capacity of knapsack bag 100. Find the optimal solution to maximize the value to weight ratio.

Item	Weight	Value
1	10	\$60
2	20	\$100
3	30	\$120
4	35	\$70
5	40	\$240

Item	Weight	Value	Value/Weight
1	10	\$60	6
2	20	\$100	5
3	30	\$120	4
4	35	\$70	2
5	40	\$240	6

Arrange in decreasing order of value/weight

Items	1	5	2	3	4
Weight	10	40	20	30	35
Value/Weight	6	6	5	4	2

→ decreasing order.

knapsack capacity, $C = 100$

Select Put for item 1 $w_1 = 10$

$$w_1 \leq C$$

$$\text{total profit} = 60$$

hence item 1 is selected to be put in knapsack bag.

item 5 $w_5 = 40$, $w_1 + w_5 < C$
 $10 + 40 < 100$

item 5 is selected

$$\text{total profit} = 60 + 240 \\ = \$300$$

Remaining capacity of bag = 50 kg.

select item 2.

$$w_1 + w_5 + w_2 \leq C \\ 10 + 40 + 20 < 100 \\ 70 < 100$$

hence item 2 is selected

$$\text{total profit} = 300 + 100 \\ = \$400$$

Remaining capacity = 30.

item 3

$$w_1 + w_5 + w_2 + w_3 = C \\ 70 + 30 = 100$$

hence item 3 is selected

$$\text{total profit} = 400 + 120 = \$520$$

Remaining capacity = 0

Solution $\rightarrow$

$$x_i = \langle 1, 1, 1, 0, 1 \rangle$$

$$\begin{aligned} \text{Total Profit} &= \sum p_i x_i \\ &= 60(1) + (100)(1) + 120(1) + 0 + 1(240) \\ &= \underline{\underline{\$520}} \end{aligned}$$

Algorithm for Fractional Knapsack

```

greedyKnapsack(P, W, M, X, n) {
    for i = 1 to n do X[i] = 0; // Initialize soln to 0
    rem_capacity = M;
    for i = 1 to n do {
        if (W(i) > rem_capacity)
            break;
        X(i) = 1;
        rem_capacity = rem_capacity - W(i);
    }
    if (i <= n)
        x[i] = (rem_capacity / W[i]);
}

```

Time Complexity $O(n \log n)$

5. b) Activity selection problem using finish time strategy. Collect the input from the user and schedule the max number of activities without clashing

start[] = {10, 12, 20, 35, 5}
 finish[] = {30, 20, 25, 50, 10}

Step 1: Sort the activities according to their finish time.

Unsorted

Activity.	A ₁	A ₂	A ₃	A ₄	A ₅
start time	10	12	20	35	5
finish time	30	20	25	50	10

Sorted Array.

Activity.	A ₅	A ₂	A ₃	A ₁	A ₄
s Start time	5	12	20	10	35
f finish time	10	20	25	30	50

Activity A₅ is selected
 selected activities = {A₅}

for A₂,

if $s_{A_2} \geq f_{A_5}$

12 > 10

true

[s = start-time
 f = finish-time]

activity A₂ is also selected

selected activities = {A₅, A₂}

for A_3

Start-time $A_3 \geq$ finish-time A_2

$$20 = 20 \quad \text{true}$$

then A_3 is also selected

selected activities = $\{A_5, A_2, A_3\}$

for A_1

Start time $A_1 \not\geq$ finish time A_5
 $10 \not\geq 25$ false.

^{not}
 A_1 activity is not selected

selected activity = $\{A_5, A_2, A_3\}$

for A_4 ,

start-time $A_4 \geq$ start-time A_5
 $35 \geq 25$

A_4 activity is selected

selected activity = $\{A_5, A_2, A_3, A_4\}$

Therefore, the max no. of activities that
 can be selected = 4

i.e., $\{A_2, A_3, A_4, A_5\}$

Diagram

k s k

5 10

a_5

✓

12 20

a_2

✓

~~20~~

20 25

a_3

✓

10 30

a_1

x

35 50

a_4

✓

not
compatible

10

20

25

35

50

Activity Selection Algorithm

Recursive-activity-selector(s, f, k, n)

$m = k + 1$

while $m \leq n$ and $s[m] < f[k]$

$m = m + 1$

if $m \leq n$

return $\{a_m\} \cup \text{Recursive-activity-selector}(s, f, m, n)$

else return $\emptyset$

6. Apply Karatsuba fast Integer multiplication algorithm using divide and conquer to find the product of the two large integers such as 5432 and 12345

Steps $\rightarrow$

divide the numbers into two halves

$$x \rightarrow x_H x_L$$

$$y \rightarrow y_H y_L \quad \text{of base } x$$

$$C_2 = x_H y_H$$

$$C_1 = (x_H + x_L)(y_H + y_L) - C_2 - C_0$$

$$C_0 = x_L y_L$$

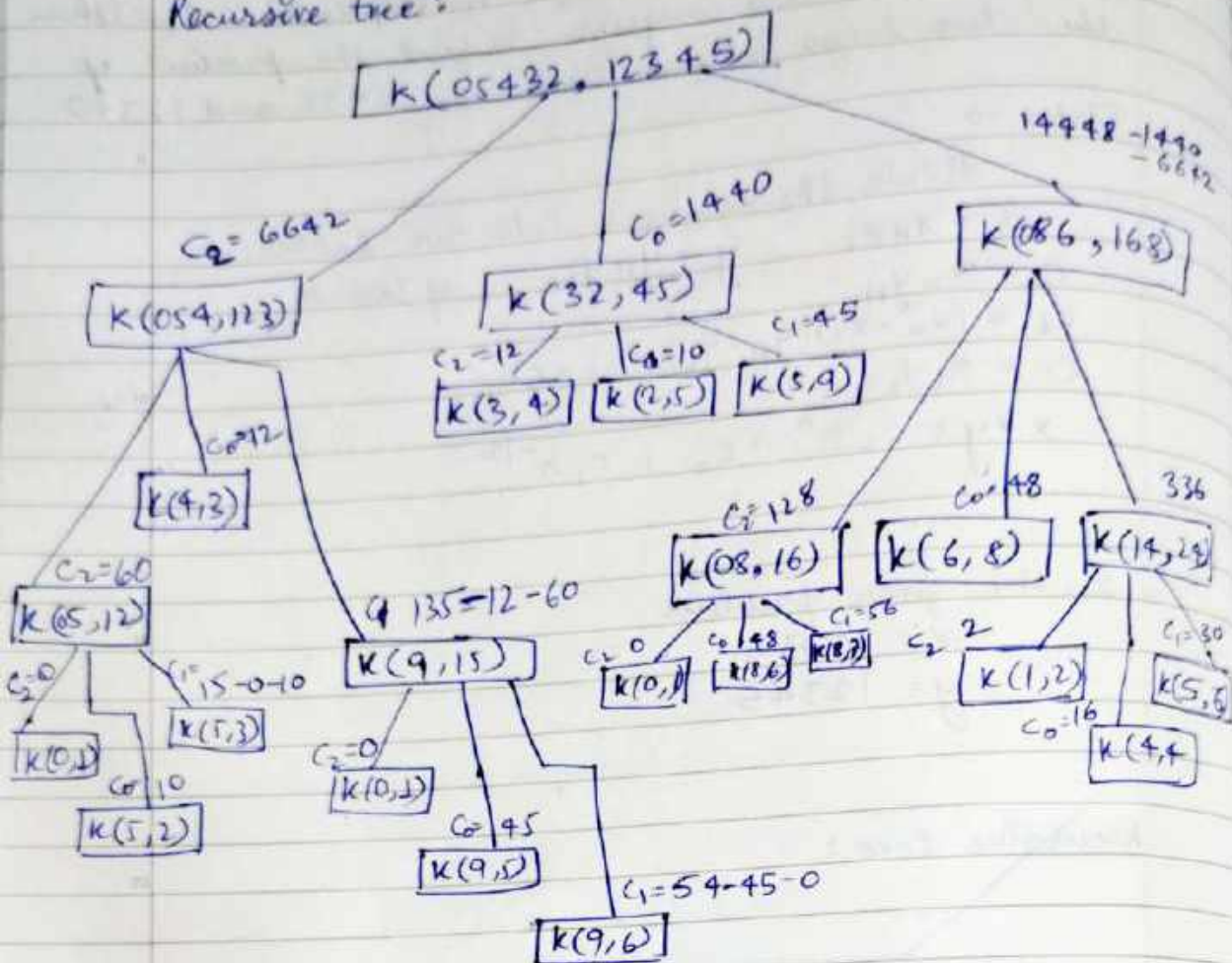
$$x * y = C_2 x^n + C_0 + C_1 x^{n/2}$$

In the given problem:

$$x = 05432$$

$$y = 12345$$

Recursive tree:



$$K(9, 15) \Rightarrow C_2 n^2 + C_1 n + C_0$$

$$C_2 = 0$$

$$C_0 = 45$$

$$C_1 = 54 - 45 - 0$$

$$C_1 = 9$$

$$n = 10$$

$$n_2 = \left\lfloor \frac{9}{2} \right\rfloor = 4$$

$$K(9, 15) = 0(10)^2 + 9(10) + 45 = 135$$

$$K(05, 12) = C_2 n^2 + C_1 n + C_0 = 0(10)^2 + 5(10) + 10 = 60$$

$$K(054, 123) = C_2 2^{2 \times n} + C_1 2^n + C_0$$

$$n = \left\lfloor \frac{\text{no. of bits}}{2} \right\rfloor = 1$$

$$= (60)(10)^2 + (35-12-60)(10)^1 + 12$$

$$= 6000 + 630 + 12$$

$$= 6642$$

$$K(32, 45) = C_2 2^{2 \times n} + C_1 2^n + C_0$$

$$\text{here } n = \left\lfloor \frac{\text{no. of bits}}{2} \right\rfloor = \left\lfloor \frac{2}{2} \right\rfloor = 1$$

$$C_2 = 12$$

$$C_0 = 10$$

$$C_1 = 45 - 12 - 10 = 12 \cdot (10)^2 + (45 - 12 - 10)(10)^1 + 10$$

$$= 23 = 1200 + 230 + 10$$

$$= 1440$$

$$K(14, 24) = C_2 2^{2 \times n} + C_1 2^n + C_0$$

$$C_0 = 16$$

$$C_1 = 30 - 16 - 2 = 14 - 2 = 12$$

$$C_2 = 2$$

$$= 2(10)^2 + 12(10) + 16$$

$$= 336$$

$$K(08, 16) = C_2 2^{2 \times n} + C_1 2^n + C_0$$

$$C_2 = 0$$

$$C_0 = 48$$

$$n = \left\lfloor \frac{\text{no. of bits}}{2} \right\rfloor = 1$$

$$C_1 = 56 - 48$$

$$C_2 = 8$$

$$= 0(10)^2 + 8(10) + 48$$

$$= 128$$

$$K(086, 168) = C_2 2^{2 \times n} + C_1 2^n + C_0$$

$$C_2 = 128$$

$$C_0 = 48$$

$$n = \left\lfloor \frac{3}{2} \right\rfloor = 1$$

$$C_1 = 336 - 128 - 48$$

$$= 160$$

$$= 128(10)^2 + 160(10) + 48$$

$$= 14448$$

$$K(05432, 12345)$$

$$n = \left\lfloor \frac{\text{no. of bits}}{2} \right\rfloor = \left\lfloor \frac{5}{2} \right\rfloor = 2$$

$$C_2 = 6642$$

$$C_0 = 1440$$

$$C_1 = 14448 - 1440 - 6642 \\ = 6366$$

$$K(05432, 12345) = C_2 x^{2 \times n} + C_1 x^n + C_0 \\ = 6642 \times (10)^4 + 6366(10)^2 + 1440$$

$$= 67058040$$

By using Karatsuba algorithm,

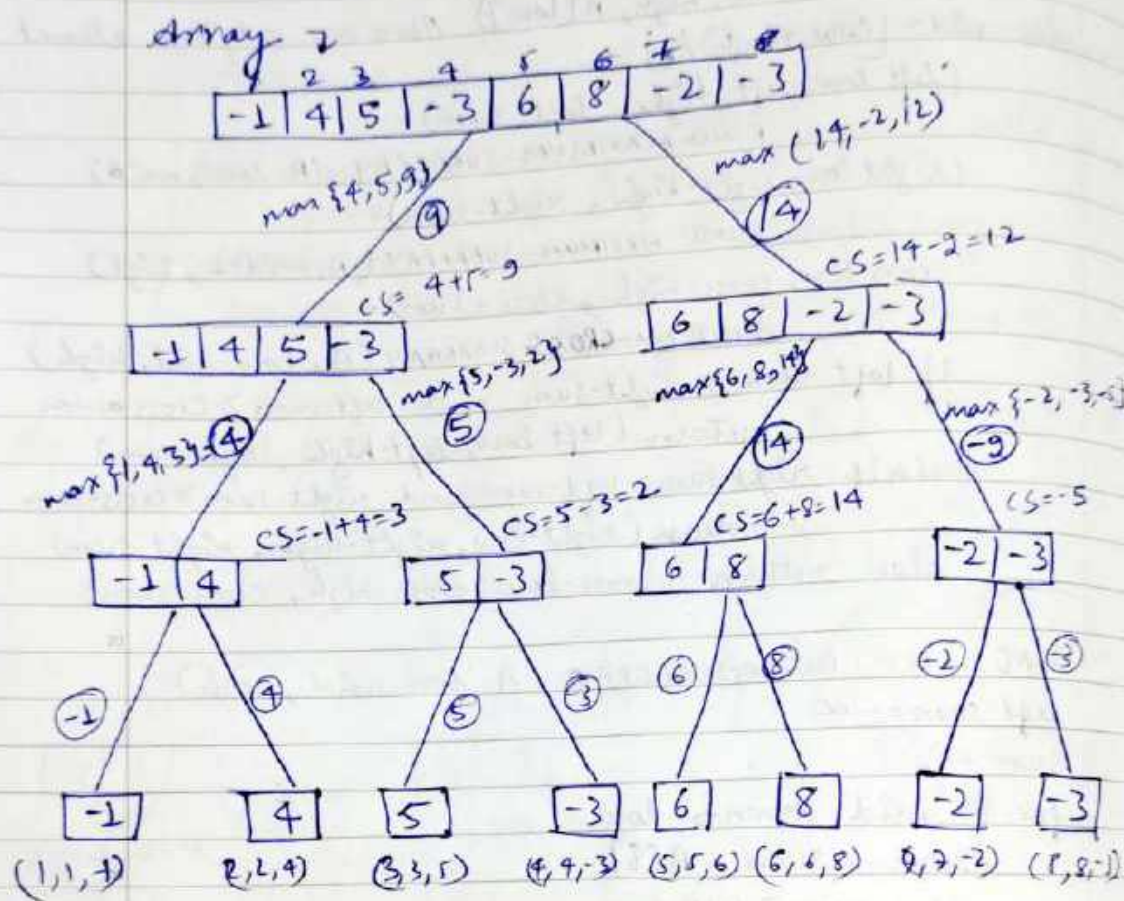
$$5432 \times 12345 = 67058040$$

Algorithm for Karatsuba

```
long karatsuba(long x, long y) {  
    if (x < 10 || y < 10)  
        return x * y;  
    int n = max(to_string(x).length(), to_string(y).length());  
    int n2 = n / 2;  
  
    long a = x / (long) pow(10, n2);  
    long b = x % (long) pow(10, n2);  
    long c = y / (long) pow(10, n2);  
    long d = y % (long) pow(10, n2);  
  
    long c2 = karatsuba(a, c);  
    long c1 = karatsuba(a + b, c + d);  
    long c0 = karatsuba(b, d);  
  
    return (long) pow(10, 2 * n2) * c2 + (long) pow(10, n2) * (c1 - c2)  
        + c0;  
}
```

Time Complexity: $T(n) = 3T(n/2) + O(n)$ 1.58
 $= O(n^{\log_2 3}) = O(n)$

7. Apply maximum subarray sum algorithm using divide and conquer to find the maximum subarray sum of the following list $[-1, 4, 5, -3, 6, 8, -2, -3]$



Calculations : LS - Left-Sum
 RS - Right-Sum

for a single item left-sum = right-sum = cross-sum

- a) for $[-1, 4]$ LSS $\rightarrow$ left Sub-Sum = $\max\{-1\} = -1$
 LS = -1
 RS = 4
 RSS $\rightarrow$ Right-Subsum = 4
 $CS = \text{Cross-Sum} = \text{LSS} + \text{RSS} = -1 + 4 = 3$
 $\text{maxSum} = \max\{-1, 4, 3\} = 4$



$$LSS = \max\{5\} = 5$$

$$RSS = \max\{-3\} = -3$$

$$LS = 5, RS = -3$$

$$CS = LSS + RSS = 5 - 3 = 2$$

$$\begin{aligned} \maxSum &= \max\{LS, RS, CS\} \\ &= \max\{5, -3, 2\} \\ &= \underline{5} \end{aligned}$$



$$LS = 4$$

$$RS = 5$$

$$LSS = \max\{4, -1 + 4\} = 4$$

$$RSS = \max\{5, 5 - 3\} = 5$$

$$\begin{aligned} CS &= LSS + RSS \\ &= 9 \end{aligned}$$

$$\begin{aligned} \maxSum &= \max\{4, 5, 9\} \\ &= \underline{9} \end{aligned}$$

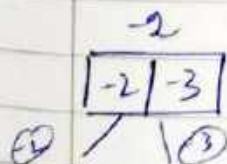


$$LS = 6, RS = 8$$

$$LSS = \max\{6\} = 6, RSS = \max\{8\} = 8$$

$$CS = LSS + RSS = 14$$

$$\begin{aligned} \maxSum &= \max\{LS, RS, CS\} = \max\{6, 8, 14\} \\ &= \underline{14} \end{aligned}$$



$$LS = -2, RS = -3$$

$$LSS = \max\{-2\} = -2$$

$$RSS = \max\{-3\} = -3$$

$$CS = LSS + RSS = -5$$

$$\begin{aligned} \maxSum &= \max\{-2, -3, -5\} \\ &= \underline{-2} \end{aligned}$$



$$LS = 14$$

$$RS = -2$$

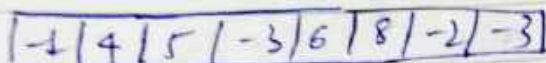
$$LSS = \max\{8, 6+8\} = 14$$

$$RSS = \max\{-2, -2-3\} = -2$$

$$CS = LSS + RSS = 12$$

$$\text{maximum sum} = \max\{LS, RS, CS\} = \max\{14, -2, 12\} = 14$$

Index $\rightarrow (5, 6, 14)$



$$LS = 9$$

$$RS = 14$$

$$LSS = \max\{-3, -3+5, -3+5+4, -3+5+4-1\} = \max\{-3, 2, 6, 5\} = 6$$

$$RSS = \max\{6, 6+8, 6+8-2, 6+8-2-3\} = \max\{6, 14, 12, 9\} = 14$$

$$CS = LSS + RSS = 6 + 14 = 20$$

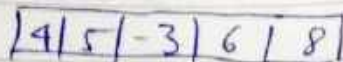
$$\text{maximum sum} = \max\{LS, RS, CS\} = \max\{9, 14, 20\} = 20$$

Index $\rightarrow (2, 6, 20)$

Maximum subarray sum = 20

Maximum subarray =

No. of elements = 5



Algorithm for Maximum Subarray.

FIND-MAXIMUM-SUBARRAY (A, low, high)

```
if (high == low)
    return (low, high, A[low]) //base case: only one element
else mid =  $\lfloor (low + high) / 2 \rfloor$ 
    (left-low, left-high, left-sum) =
        FIND-MAXIMUM-SUBARRAY (A, low, mid)
    (right-low, right-high, right-sum) =
        FIND-MAXIMUM-SUBARRAY (A, mid+1, high)
    (cross-low, cross-high, cross-sum) =
        FIND-MAX-CROSS-SUBARRAY (A, low, mid, high)
    if left-sum  $\geq$  right-sum and left-sum  $\geq$  cross-sum
        return (left-low, left-high, left-sum)
    else if right-sum  $\geq$  left-sum and right-sum  $\geq$  cross-sum
        return (right-low, right-high, right-sum)
    else return (cross-low, cross-high, cross-sum)
```

FIND-MAX-CROSSING-SUBARRAY (A, low, mid, high)

left-sum = $-\infty$

sum = 0

for $i = \text{mid}$ down to low

sum = sum + A[i]

if sum > left-sum

left-sum = sum

max-left = i

right-sum = $-\infty$

sum = 0

for $j = \text{mid}+1$ to high

sum = sum + A[j]

if sum > right-sum

right-sum = sum

max-right = j

return (max-left, max-right, left-sum + right-sum)

Time Complexity = $T(n) = 2T(n/2) + O(n) = O(n \log n)$

8. Given the following 2 sequences such as $\{A, S, D, F, G, P\}$ and $\{A, D, F, G, R\}$, find the longest common subsequence between the 2 sequences using dynamic programming strategy.

Let $X = \{A, D, F, G, R\}$
 $Y = \{A, S, D, F, G, P\}$

$c(i, j)$		Y						
			A	S	D	F	G	P
X		0	1	2	3	4	5	6
0		0	0	0	0	0	0	0
A	1	0	①	← 1	← 1	← 1	← 1	← 1
D	2	0	1 ↑	1 ↑	②	← 2	← 2	← 2
F	3	0	1 ↑	1 ↑	2 ↑	③	← 3	← 3
G	4	0	1 ↑	1 ↑	2 ↑	3 ↑	④	← 4
R	5	0	1 ↑	1 ↑	2 ↑	3 ↑	4 ↑	4 ↑

$$c[i, j] = \begin{cases} 0 & i=0 \text{ or } j=0 \\ c[i-1, j-1] + 1 & i, j > 0 \text{ and } x_i = y_j \\ \max(c[i, j-1], c[i-1, j]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j \end{cases}$$

longest subsequence = ADFG

Algorithm for longest common subsequence.

LCS.length(x, y)

m = x.length

n = y.length

let b[1..m, 1..n] and c[0..m, 0..n]

for i = 1 to m

c[i, 0] = 0

for j = 0 to n

c[0, j] = 0

for i = 1 to m

for j = 1 to n

if $x_i = y_j$

c[i, j] = c[i-1, j-1] + 1

b[i, j] = "↖"

else if c[i-1, j] >= c[i, j-1]

c[i, j] = c[i-1, j]

b[i, j] = "↑"

else c[i, j] = c[i, j-1]

b[i, j] = "←"

return c and b.

PRINT-LCS(b, x, i, j)

if i == 0 or j == 0

return

if b[i, j] == "↖"

PRINT-LCS(b, x, i-1, j-1)

print x[i]

else if b[i, j] == "↑"

PRINT-LCS(b, x, i-1, j)

else PRINT-LCS(b, x, i, j-1)

9. Solve the 0/1 Knapsack Problem using Dynamic Programming for given inputs.

Input: $val[] = \{6, 10, 12, 18, 24\}$
 $wt[] = \{1, 5, 2, 3, 4\}$
 $W = 6$

$W = 6$

$n = 5$ (no. of items)

$n \backslash w$		$j \rightarrow$							
		0	1	2	3	4	5	6	
v_i	w_i	0	0	0	0	0	0	0	
6	1	1	0	6	6	6	6	6	
10	5	2	0	6	6	6	10	16	
12	2	3	0	6	12	18	18	18	
18	3	4	0	6	12	18	24	30	
24	4	5	0	6	12	18	24	30	36

$i \rightarrow$ no. of items available

$j \rightarrow$ no. of capacity in knapsack bag

$F[i, j] \rightarrow$ optimal profit obtained for (i, j)

$$F(i, j) = \begin{cases} \max \{F(i, j), v_i + F(i-1, j-w_i)\} & \text{when } j-w_i \geq 0 \\ F(i-1, j) & j-w_i < 0 \end{cases}$$

$$F(0, j) = F(i, 0) = 0$$

$$f(1,1) = j=1 \quad w_i=1 \quad v_i=6$$

$$j-w \leq 0 \\ 1-1=0$$

$$f(1,1) = \max \{ f(i-1, j), v_i + f(i-1, j-w_i) \}$$

$$\max \{ f(0,1), 6 + f(0,0) \} \\ \max \{ 0, 6 \} \\ = 6$$

$$f(1,1) = 6$$

$$f(1,2) = j=2 \quad w_i=1 \\ = \max \{ f(0,2), 6 + f(0,1) \} \\ \max \{ 0, 6 \} \\ = 6$$

$$f(1,3) = j=3 \quad w_i=1 \quad j \geq w_i \\ \max \{ f(0,3), 6 + f(0,2) \} \\ = \max \{ 0, 6 \} \\ = 6$$

Similarly $f(1,4) = 6$
 $f(1,1)$

$$f(1,5) = \max \{ f(0,5), 6 + f(0,4) \} \\ = 6$$

$$f(1,6) = \max \{ f(0,6), 6 + f(0,5) \} \\ = 6$$

$$f(2,1) = \begin{matrix} w_i = 5 & j = 1 \\ w_i > j \end{matrix}$$

$$f(2,1) = f(i-1, j) = f(1,1) = 6$$

$$f(2,2) \quad \begin{matrix} w_i = 5, & j = 2 \\ w_i > j \end{matrix}$$

$$f(2,2) = f(1,2) = 6$$

$$f(2,3) = \begin{matrix} w_i > j \end{matrix}$$

$$f(2,3) = f(1,3) = 6$$

$$f(2,4) = \begin{matrix} w_i > j \end{matrix}$$

$$f(2,4) = f(1,4) = 6$$

$$f(2,5) = \begin{matrix} w_i = 5 & j = 5 \\ j - w_i > 0 \\ 0 > 0 \end{matrix}$$

$$f(2,5) = \max \{ f(1,5), w_i + f(1,0) \}$$

$$= \max \{ 6, 10 + 0 \}$$

$$= 10$$

$$f(2,6) \rightarrow \begin{matrix} j = 6 & j > w_i \end{matrix}$$

$$f(2,6) = \max \{ f(1,6), 10 + f(1,1) \}$$

$$\max \{ 6, 16 \}$$

$$= 16$$

$$f(3,1) = \begin{matrix} w_i = 2 & j = 1 \\ w_i > j \end{matrix}$$

$$f(3,1) = f(2,1)$$

$$f(3,2) = \begin{matrix} j - w_i = 0 \end{matrix}$$

$$f(3,2) = \max \{ f(2,2), 12 + f(2,0) \}$$

$$= \max \{ 6, 12 \}$$

$$= 12$$

$$f(3,3) = j - w_i = 3 - 2 = 1 > 0$$

$$f(3,3) = \max \{ f(2,3), 12 + f(2,1) \}$$

$$= \max \{ 6, 12 + 6 \}$$

$$= 18$$

$$f(3,4) = j - w_i = 4 - 2 = 2 > 0$$

$$f(3,4) = \max \{ f(2,4), 12 + f(2,2) \}$$

$$= \max \{ 6, 12 + 6 \}$$

$$= 18$$

$$f(3,5) = j - w_i = 5 - 2 = 3 > 0$$

$$f(3,5) = \max \{ f(2,5), 12 + f(2,3) \}$$

$$= \max \{ 10, 12 + 6 \}$$

$$= 18$$

$$f(3,6) = j - w_i = 6 - 2 = 4 > 0$$

$$f(3,6) = \max \{ f(2,6), 12 + f(2,4) \}$$

$$= \max \{ 16, 18 \}$$

$$= 18$$

$$f(4,1) = w_i = 3 \quad j = 1 \quad v_i = 18$$

$$w_i > 1$$

$$f(4,1) = f(3,1) = 6$$

$$f(4,2) = j - w_i < 0$$

$$f(4,2) = f(3,2)$$

$$= 12$$

$$f(4,3) = j - w = 0 \quad \checkmark$$

$$f(4,3) = \max \{ f(3,3), 18 + f(3,0) \}$$

$$= \max \{ 18, 18 \}$$

$$= 18$$

27 BCE 0756

Date: _____

$$F(4,4) = j - w_i = 1$$

$$F(4,4) = \max \{ F(3,4), 18 + F(3,1) \}$$

$$\max \{ 18, 24 \}$$

$$= 24$$

$$F(4,5) = j - w_i = 2$$

$$F(4,5) = \max \{ F(3,5), 18 + F(3,2) \}$$

$$= \max \{ 18, 18 + 12 \}$$

$$= 30$$

$$F(4,6) = j - w_i = 3$$

$$F(4,6) = \max \{ F(3,6), 18 + F(3,3) \}$$

$$= \max \{ 18, 18 + 18 \}$$

$$= 36$$

$$F(5,1) = j - w_i = 4 \quad v_i = 24$$

$$j - w_i = -3 < 0$$

$$F(5,1) = F(4,1)$$

$$= 6$$

$$F(5,2) = j - w_i = -2 < 0$$

$$F(5,2) = F(4,2)$$

$$= 12$$

$$F(5,3) = j - w_i = -1 < 0$$

$$F(5,3) = F(4,3)$$

$$= 18$$

$$F(5,4) = j - w_i = 0$$

$$= \max \{ F(4,4), 24 + F(4,0) \}$$

$$\max \{ 24, 0 \}$$

$$= 24$$

$$f(5, 5) = j - w_i = 1$$

$$\max \{ f(4, 5), 24 + f(4, 1) \}$$
$$\max \{ 30, 24 + 6 \}$$
$$= 30$$

$$f(5, 6) = j - w_i = 2$$

$$\max \{ f(4, 6), 24 + f(4, 2) \}$$

$$= \max \{ 36, 24 + 12 \}$$

$$= 36$$

Now selecting items,

$$F(5,6) = 36$$

$$\text{check } F(i,j) > F(i-1,j) \\ F(5,6) \neq F(4,6) \quad \text{false}$$

7th item is not added

$$F(i,j) > F(i-1,j) \\ F(4,6) > F(3,6) \quad \text{true} \\ 36 > 18$$

4th item is added

Move from $F(i,j)$ to $F(i-1, j-1)$

$$F(i,j) \rightarrow F(i-1, j-w_i)$$

$$F(4,6) \rightarrow F(3, 6-w_5)$$

$$F(4,6) \rightarrow F(3, 6-3) = F(3,3)$$

$$F(3,3) > F(2,3) \\ 18 > 6$$

Hence. 3rd item is added

$$\text{Move } F(3,3) \xrightarrow{+} F(2, 3-w_3)$$

$$F(3,3) \rightarrow F(2, 3-2)$$

$$F(2,1) \neq F(1,1) \\ 6 > 6$$

Not true.

Hence 2nd item is not added.

$$F(1,1) > F(0,1) \\ 6 > 0$$

1st item is added.

$$\text{Sol}^n = \langle x_1 \quad x_2 \quad x_3 \quad x_4 \quad x_5 \\ 1 \quad 0 \quad 1 \quad 1 \quad 0 \rangle$$

Algorithm for 0/1 Knapsack

MFKnapsack (i, j)

if $F[i, j] < 0$

if $j < \text{weight}[i]$

value $\leftarrow$ MFKnapsack($i-1, j$)

else

value $\leftarrow$ max(MFKnapsack($i-1, j$),

values[i] + MFKnapsack($i-1, j - \text{weight}[i]$))

$F[i, j] \leftarrow$ value

return $F[i, j]$

10. Determine where to parenthesize a chain of matrices so that finding their product will require only min no. of scalar multiplication. For example, let the 5 matrices be $A (5 \times 3)$ $B (3 \times 4)$, $C (4 \times 2)$ $D (2 \times 5)$ $E (5 \times 2)$

$$A \times B \times C \times D \times E$$

$5 \times 3 \quad 3 \times 4 \quad 4 \times 2 \quad 2 \times 5 \quad 5 \times 2$

$$P_0 \quad P_1 \quad P_2 \quad P_3 \quad P_4 \quad P_5$$

$5 \quad 3 \quad 4 \quad 2 \quad 5 \quad 2$

m

	1	2	3	4	5
1	0	60	54	104	90
2		0	24	54	60
3			0	40	36
4				0	20
5					0

$$m(i, j) = \begin{cases} 0 & \text{if } i=j \\ \min_{i \leq k < j} \{ m(i, j) + m(i+1, k) + P_{i-1} P_k P_j \} & i < j \end{cases}$$

$$m(1,1) = m(2,2) = m(3,3) = m(4,4) = m(5,5) = 0$$

$$m(1,2) = \begin{cases} k=1 & m(1,1) + m(2,2) + P_0 P_1 P_2 \\ & = 5 \times 3 \times 4 = 60 \end{cases}$$

$$m(2,3) = \begin{cases} k=2 & = m(2,2) + m(3,3) + P_1 P_2 P_3 = 3 \times 4 \times 2 = 24 \end{cases}$$

$$m(3,4) = \begin{cases} k=3 & m(3,3) + m(4,4) + P_2 P_3 P_4 \\ & 4 \times 2 \times 5 = 40 \end{cases}$$

$$m(4,5) = \begin{cases} k=4 & m(4,4) + m(5,5) + P_3 P_4 P_5 = 2 \times 5 \times 2 \\ & = 20 \end{cases}$$

$$m(1,3) = \begin{cases} k=1 & m(1,1) + m(2,3) + P_0 P_1 P_3 = 24 + 30 = 54 \\ \min & k=2 & m(1,2) + m(3,3) + P_0 P_2 P_3 = 60 + 40 = 100 \end{cases}$$

$$m(2,4) = \min \begin{cases} k=2 & m(2,2) + m(3,4) + P_L P_2 P_r = 40 + 60 = 100 \\ k=3 & m(2,3) + m(4,4) + P_L P_3 P_r = 24 + 30 = \textcircled{54} \end{cases}$$

$$m(3,5) = \min \begin{cases} k=3 & m(3,3) + m(4,5) + P_L P_3 P_5 = 20 + 16 = \textcircled{36} \\ k=4 & m(3,4) + m(5,5) + P_L P_4 P_5 = 40 + 40 \end{cases}$$

$$m(1,4) = \min \begin{cases} k=1 & m(1,1) + m(3,4) + P_0 P_1 P_r = 54 + 75 \\ k=2 & m(1,2) + m(3,4) + P_0 P_2 P_r = 60 + 90 + 100 \\ k=3 & m(1,3) + m(4,4) + P_0 P_3 P_r = 54 + 50 = \textcircled{104} \end{cases}$$

$$m(2,5) = \min \begin{cases} k=2 & m(2,2) + m(3,5) + P_L P_2 P_5 = 36 + 29 = \textcircled{60} \\ k=3 & m(2,3) + m(4,5) + P_L P_3 P_5 = 24 + 40 + 12 = 76 \\ k=4 & m(2,4) + m(5,5) + P_L P_4 P_5 = 54 + 30 = 84 \end{cases}$$

~~$m(3,6)$~~

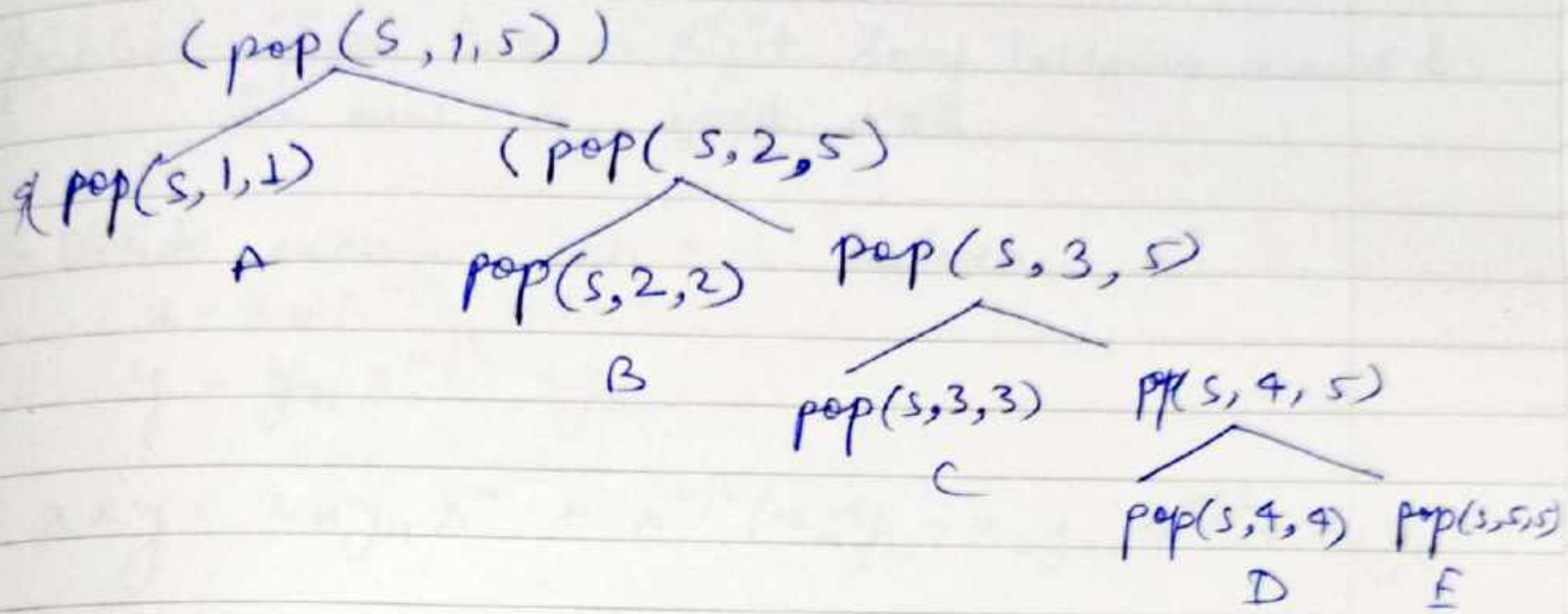
$$m(1,6) = \min \begin{cases} k=1 & m(1,1) + m(3,5) + P_0 P_1 P_r = 60 + 30 = \textcircled{90} \\ k=2 & m(1,2) + m(3,5) + P_0 P_2 P_r = 60 + 36 + 40 = 136 \\ k=3 & m(1,3) + m(4,5) + P_0 P_3 P_r = 54 + 20 + 20 = 94 \\ k=4 & m(1,4) + m(5,5) + P_0 P_4 P_r = 104 + 50 = 154 \end{cases}$$

k	1	2	3	4	5
1	x	1	1	1	1
2		x	2	3	2
3			x	3	3
4				x	4
5					x

~~((A(B(C(D(E))))))~~ ~~(A)(B)(C)(D)(E))~~

→ ((A)(B(C(DE))))

using parenthesis tree .



(A(B(C(D(E)))))

Algorithm

Matrix-chain-order(p) ^{→ dimension matrix}

$n = p.length - 1$

let $m[1 \dots n, 1 \dots n]$ and $s[1 \dots n]$ be new tables

for $i = 1$ to n

$m[i, i] = 0$

for $l = 2$ to n // l is length of chain

for $i = 1$ to $n+1-l$

$j = i + l - 1$

$m[i, j] = \infty$

for $k = i$ to $j-1$

$q = m[i, k] + m[k+1, j] + p_i + p_k p_j$

if $q < m[i, j]$

$m[i, j] = q$

$s[i, j] = k$

return m , and s .

print-optimal-parens(s, i, j)

if $i == j$

print " A_i "

else print "("

print-optimal-parens($s, i, s[i, j]$)

print-optimal-parens($s, s[i, j] + 1, j$)

print ")"

Time Complexity = $O(n^3)$

Space Complexity = $O(n^2)$